Solución de Problemas

La solución de problemas tiene que ver con aquellos que los humanos resuelven en su vida diaria. Aunque los dominios son diferentes y los métodos también, la tecnología de la IA proporciona un conjunto de formalismos para representar las técnicas y los problemas en su proceso para resolverlos.

Algunos de los problemas en la vida real son difíciles debido a:

- El número de soluciones posibles en el espacio de búsqueda es tan grande, que es prohibitivo utilizar la búsqueda exhaustiva como la mejor respuesta.
- El problema es tan complicado que el solo facilitar cualquier respuesta, implicaría usar modelos simples que darían resultados poco útiles.
- La función de evaluación que describe la calidad de cualquier solución propuesta, es ruidosa y varía con el tiempo, por lo que se requiere no solo la propuesta de una solución sino un conjunto de ellas.
- Las posibles soluciones están tan fuertemente restringidas que, el construir una sola es difícil, lo que lleva solo a la consideración de la búsqueda a una solución óptima.
- La persona que intenta resolver el problema esta inadecuadamente preparada o se imponen barreras psicológicas que le impiden visualizar una solución.

Áreas típicas que requieren razonamiento complejo para solucionar problemas:

- Planeación: Habilidad para decidir sobre una buena secuencia de acciones para lograr la metas.
- Visión: Habilidad para tomar conciencia de lo que vemos.
- Robótica: Habilidad para moverse y actuar en el mundo, posiblemente respondiendo a nuevas percepciones.
- Lenguaje Natural: Habilidad para comunicarse con otros.
- Razonamiento en general.

Áreas típicas que requieren de destreza o entrenamiento (experticia):

- Diagnóstico médico.
- Reparación de equipo.
- Configuración de computadoras.
- Análisis financiero.
- Exploración geológica y submarina.
- Reconocimiento de imágenes.
- Sistemas educativos.
- Sistemas de predicción.
- Mantenimiento de redes.
- Tutores inteligentes.
- Asesorías legales

- Etc.

Objetivos centrales de la IA:

a) Hacer que las computadoras sean más útiles.

- Programas inteligentes para diversas tareas.
- Sistemas expertos y sistemas basados en conocimiento.

b) Entender los principios que hacen posible la inteligencia en general, por medio de la inteligencia programada.

En la IA se origino la idea de solución de problemas con el objetivo de diseñar un sistema con la capacidad de resolver y reconocer problemas de cualquier tipo. A la fecha tal sistema no existe. Lo que ha habido es un gran progreso en lo concerniente a la modelación de problemas y al desarrollo de estrategias.

La solución de problemas acompaña a todo lo relacionado con ciencias de la computación porque cualquier tarea computacional puede considerarse como un problema por resolver.

Inicialmente los problemas de juegos (ajedrez, damas, acertijos, etc.) se elegían, no porque pareciesen simples o claros; sino porque dentro de sus pequeñas estructuras, proporcionan complejidades que pueden mapearse a problemas reales.

Pasos en la solución de problemas:

1. Formular el objetivo en base a la situación actual.
2. Formular el problema.

- Espacio de estados en donde se satisface el objetivo
- Acciones que pueden causar transiciones entre estados de ese mundo o espacio.
- Que clase de esas acciones se deben considerar para alcanzar el estado objetivo (abstracción para convertir un problema intratable en tratable)

En suma es el proceso de decidir que acciones y estados considerar que van a servir para llegar al objetivo.

3. Usar búsqueda analizando las posibles secuencias de acciones, basadas en información de opciones intermedias, que guíen a estados de valor conocido y elegir el mejor de estos.
4. Obtención de la solución mediante la secuencia de acciones resultante del algoritmo que toma como entrada, un problema.

5. Ejecutar las acciones recomendadas.

Entonces se definen los conceptos siguientes:

Problema

Es una colección de información que el programa usa para decidir que hacer (dentro de dicha información se tienen los estados y las acciones).

Estado inicial

Es el estado donde el programa mismo sabe que está en él.

Operador

Es el conjunto de acciones disponibles al programa: reglas de inferencia (Modus ponens, resolución, etc.), movimientos legales (ajedrez, damas, gato, etc.), substituciones válidas (integración por partes, etc.), restricciones, etc.

Espacio de estados

Es el conjunto de todos los estados factibles de ser alcanzados desde un estado inicial, por medio de una secuencia de acciones.

Trayectoria

Es una secuencia de acciones, dentro del espacio de estados, que guían de un estado a otro.

Prueba del objetivo

Procedimiento que se aplica para revisar si un estado determinado es el objetivo.

Costo de la Trayectoria

Es una función que asigna un costo a una trayectoria. Esta entonces será la suma de todos los costos individuales a lo largo de la trayectoria.

Solución

Es la salida de un algoritmo consistente en la trayectoria desde el estado inicial, al estado que satisface la prueba del objetivo.

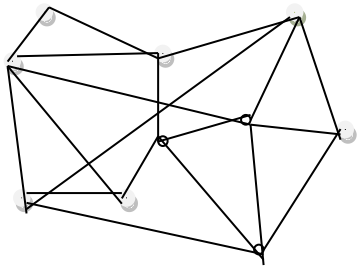
Algunos problemas parecerían ser más fáciles ~~que~~ el **SAT** debido que ellos sugieren una función de evaluación (para ver que falta o si se llegó al objetivo) de manera natural. Aún así el tamaño de espacio de búsqueda podría ser enorme.

El problema del SAT (satisfacibilidad booleana). Given a Boolean expression **E,** decide if there is some assignment to the variables in **E** such that **E** is true. O sea una asignación a cada variable x_i para

$i = 1,\ldots,n$

Por ejemplo en el caso de 100 variables dada la expresión en forma normal conjuntiva:

$E(x) = (x_{17} \vee \neg x_{37} \vee x_{73}) \wedge (\neg x_{11} \vee \neg x_{56}) \wedge \ldots \wedge (x_2 \vee x_{43} \vee \neg x_{77} \vee \neg x_{89} \vee \neg x_{97})$ 2^{100} posibilidades



Considere el problema del TSP.

La distancia en cualquier par de ciudades *i, j* es la misma en cualquier dirección, o sea,

$dist(i, j) = dist(j, i)$ este es el caso simétrico.

Para el caso asimétrico seria $dist(i, j) \neq dist(j, i)$

¿Cual es el espacio de búsqueda para este problema?

Si lo vemos como un conjunto de permutaciones de *n* ciudades, entonces cualquier permutación nos da una lista ordenada por ser visitadas a partir de la ciudad de inicio.

Si fuesen 20 ciudades entonces los "tours" podrían ser:

2-...-6-15-3-11-19-17,

15-3-11-19-17-2-…-6,

3-11-19-17-2-…-6-15, etc.

Como hay n! modos de permutar n números entonces,

$|S| = n!/n$ para el caso asimétrico, y $|S| = (n-1)!/2$ para el caso simétrico.

Comparando SAT y TSP, para cualquier número $n > 6$, el número de posibles soluciones en el TSP es mayor que en el SAT.

Ejemplo: si $n = 6$, en TSP hay $5!/2 = 60$ soluciones diferentes y para el SAT hay $2^6 = 64$ soluciones. Pero para $n = 7$ entonces hay 360 y 128 respectivamente.

Respecto a modelar un problema

Ejemplo:

Existe una empresa con número determinado de almacenes de ciertos productos (resmas de papel), y cuenta además con centros de distribución.

n = fuentes, k = destinos,

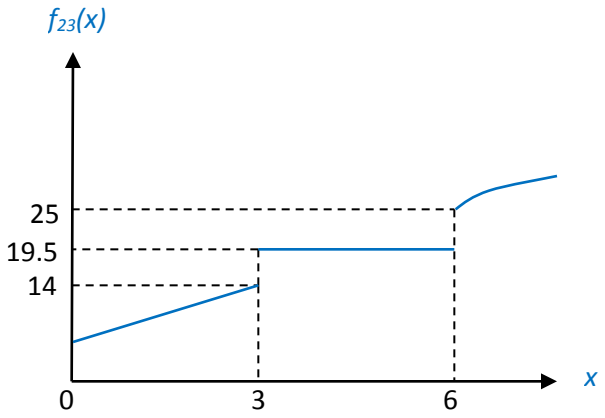
Hay un costo de entrega entre cada fuente i y destino j determinado por la función $f_{ij.}$ La configuración de esta función depende de varios factores: distancia, calidad del camino, tráfico, altos, velocidad límite promedio, etc.

Supongamos que la función anterior entre la fuente 2 y el destino 3 se define como:

$$f_{23}(x) = \begin{cases} 0 & \text{if } x = 0 \\ 4 + 3.33x & \text{if } 0 < x \leq 3 \\ 19.5 & \text{if } 3 < x \leq 6 \\ 0.5 + 10\sqrt{x} & \text{if } 6 < x \end{cases}$$

donde x se refiere a la cantidad de resmas transportadas desde i a j

(ver figura)



Si no hay entrega el costo es cero. Si se cargan tres resmas se suponen cuatro unidades por el contenedor más 3.33 unidades por cada resma y así el costo se incrementa linealmente. Si transportamos más de tres y menos de seis se puede usar una sola malla especial y en este caso se tiene un costo uniforme independientemente del número de resmas. Pero si transportamos más de seis se tiene que usar una gran caja reforzada y el costo depende del número de resmas y crece como la raíz cuadrada de esa cantidad más un factor pequeño de 0.5 unidades.

Con esas premisas se puede construir el modelo que implica:

Minimizar $\sum_{i=1}^{n} \sum_{j=1}^{k} f_{ij}(x_{ij})$,

sujeta a $\sum_{j=1}^{k} x_{ij} \leq fuente(i), para\ \ i = 1, 2, \dots, n,$

$$\sum_{i=1}^{n} x_{ij} \geq destino(j), para\ \ j = 1, 2, \dots, k$$

$$x_{ij} \geq 0, para\ \ i = 1, 2, \dots, n\ \ y\ \ j = 1, 2, .., k,$$

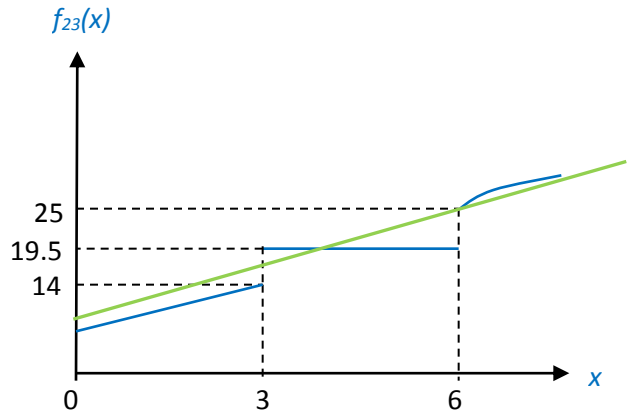
Esas restricciones expresan:

- No hay transporte de cualquier fuente que exceda el número de resmas disponibles en esa fuente.
- El transporte total a cualquier destino debe satisfacer su demanda (ej. El transporte total debe ser al menos igual al numero de resmas ordenadas)

.

Si queremos derivar una solución, podemos usar una aproximación del modelo con una recta, o sea,

$f'_{23}(x)=2.66x + 8.25$



O usar el modelo como esta y aplicar un modelo no tradicional de optimización como el templado simulado (simulated annealing) o la programación evolutiva (evolutionary programming).

De estos dos enfoques se desprende que:

Problema ⇨ Modelo aproximado ⇨ Solución precisa (del modelo aproximado).

Problema ⇨ Modelo preciso ⇨ Solución aproximada (del modelo preciso).

Problemas y su complejidad.

Dentro de la solución de problemas, los algoritmos juegan una parte medular ya que es deseable que sean los más rápidos involucrando con ello el *tiempo*, siendo este una variable dentro del tamaño de la instancia de un problema. La instancia refleja la cantidad necesaria de datos de entrada.

La dificultad de una instancia de un problema varía con su tamaño, y frecuentemente se mide de manera informal. En el TSP por ejemplo se mide con el número de ciudades, pero hay otros parámetros que también contribuyen (distancias, los gastos, trafico, etc.) a los datos de entrada de tal instancia.

Formalmente una instancia podría medirse en base el esquema de codificación que cada problema asocia a sus entradas.

La longitud de una instancia I de un problema π se define por el número de símbolos usados para describir I. Dicho número se obtiene del esquema de codificación para π.

Ejemplo: si tenemos m ciudades, su conjunto finito sería,

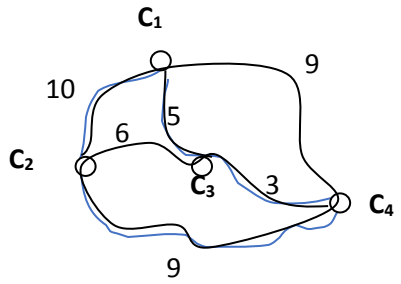
$C = \{c_1, c_2, c_3, c_4, \ldots, c_m\}$

Y por cada par c_i, c_j la distancia $d(c_i, c_j)$.

Una solución es un ordenamiento de ciudades que minimice:

$$[\sum_{i=1}^{m-1} d(c_i, c_{i+1})] + d(c_m, c_1)$$

Si fuese el esquema representado:



Una instancia del problema sería dada por C = {c_1, c_2, c_3, c_4} y el ordenamiento <c_1, c_2, c_4, c_3> es una solución para esa instancia con longitud = 27

El alfabeto que se podría utilizar sería:

{*c, [,], /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9*} el cual siendo codificado por la cadena,

"c[1]c[2]c[3]c[4]//10/5/9//6/9//3" ,

representa una entrada de *longitud = 32*

Para la determinación de la complejidad en tiempo, se proporciona la más grande que se ocupa para resolver el problema. En esta complejidad intervienen la *longitud de la entrada* y la del *tiempo de ejecución.*

Los algoritmos de complejidad de tiempo polinomial se definen como aquellos que tienen una función *O(p(n))* de acuerdo a una función polinomial *p*, y *n* igual a la longitud de la entrada.

Cualquier algoritmo que no esté de esta manera acotado se le considera con expansión de tiempo exponencial aunque no sea considerado exactamente como tal, como $n^{\log n}$.

Para darnos cuenta de la magnitud de algunos problemas podemos ver la siguiente tabla:

Se expresan tiempos de ejecución.

Función de complejidad de tiempo	Tamaño de ***n***					
	10	20	30	40	50	60
n	.00001 s	.00002 s	.00003 s	.00004 s	.00005 s	.00006 s
n^2	.0001 s	.0004 s	.0009 s	.0016 s	.0025 s	.0036 s
n^3	.001 s	.008 s	.027 s	.064 s	.125 s	.216 s
n^5	.1 s	3.2 s	24.3 s	1.7 m	5.2 m	13.0 m
2^n	.001 s	1 s	17.9 s	12.7 días	35.7 años	366 centurias
3^n	.059 s	58 m	6.5 años	3855 años	$2x10^8$ centurias	$1.3x10^{13}$ centurias

Por eso siempre es deseable llegar a obtener algoritmos de tiempo polinomial. Muchos de estos son solo variaciones de la búsqueda exhaustiva.

Un problema es *intratable* si es muy difícil (duro) que un algoritmo de tiempo polinomial pueda resolverlo.

Algunas veces un algoritmo exponencial puede ser más eficiente que uno polinomial cuando el tamaño de n es limitado.

Heurística

Heuriskein - descubrir

Estudio de métodos utilizados para descubrir o inventar técnicas para la solución de problemas (Polya 1957).

Búsqueda heurística:

- No se garantiza que llegue a la solución
- No se garantiza la calidad de la solución

Las técnicas (corazonadas) las usaban los expertos para solucionar problemas sin el uso de la búsqueda exhaustiva (HPP de Newell & Simons).

Actualmente:

Es un adjetivo que se refiere a cualquier técnica que mejora el rendimiento (en el caso promedio) de una tarea en la solución de problemas, pero no necesariamente mejora el rendimiento del caso peor.

Dentro del área especifica de algoritmos de búsqueda:

"Función que mejora un estimado del costo de la solución"

En la programación heurística normalmente no existe una vía sistemática construir una solución. Pero aplicando experiencia se puede desarrollar una solución.

Técnicas generales de la heurística

- La generalización: Es una extensión de métodos para casos más generales.
- La relajación: Se modifica un problema en base a la omisión de algo. La solución del problema nuevo sirve como ayuda para resolver el problema original.
- La analogía: Se trata problemas semejantes por métodos semejantes, pero es difícil explicar que es semejante en cada cosa.

Adicional a lo anterior hay que considerar unas clases de problemas:

Problemas con descripciones formales.

- Clase sin soluciones por computadora
- Clase que puede tener soluciones por computadora

  Problemas tratables: el número de pasos del algoritmo de solución, crece en forma polinomial con el tamaño del problema actual. A estos problemas se les llama del tipo P.

  Problemas intratables: Aquí el número de pasos del algoritmo crece mas rápido que lo polinomial, por ejemplo exponencial. A estos problemas se les llama del tipo NP (*Non-deterministic Polyinomial*). En estos problemas no se conoce un algoritmo polinomial de solución, pero si es posible comprobar en tiempo polinomial, si su costo es mejor que un valor determinado.

~~function SummATIGN(svience) returns a number~~

~~*sum* —0~~

~~**for *i* = 1 to** LENom(sequence) do~~

~~.*sum* q— *sum* + *sequence[i]*~~

~~return *sum*~~

Por ejemplo: el problema de la mochila es NP puesto que, para una instancia pequeña ($n = 3$), la suma de dos números y su comparación nos indican si hay por ejemplo ¿una solución con $costo < 10$?

El problema de la mochila.

Seleccionar de entre un conjunto de n productos, cada uno con valor c_i y un volumen v_i aquellos que quepan en un recipiente con volumen V, y que tengan el mayor valor posible.

Determinar un subconjunto: I* del conjunto {1…n} para el cual

$$\sum_{I^*} c_i = \max_{I \subseteq \{1 \ldots n\}} \sum_I c_i$$

con la restricción

$$\sum_{I^*} V_i \leq V$$

Si tenemos una instancia concreta de un problema, por ejemplo:

n=3, c_1=5, c_2=4, c_3=7, v_1=2, v_2=1, v_3=3, V=5

Se puede decidir en tiempo polinomial si la solución I={1,3} tiene un costo mayor de 10, haciendo solo una suma de dos números y una combinación,

Pero a medida que las instancias se incrementan entonces el problema se complica.

En general en este problema el número de subconjuntos del conjunto {1…n} es 2^n.

Si un CPU, en un segundo generase un millón de esos subconjuntos y evaluar su valor, se necesitaría un segundo para hallar la solución si n=20 (ya que 2^{20} = 1,048,580), pero dos semanas para n=40 y 365 siglos cuando n=60.

Formulación de problemas de tipo *juego.*

Robot que aspira

- *Estados*: El agente esta en uno de los dos lugares, cada uno de los cuales puede o no puede tener basura. Hay

  $2x2^2$=8 posibles estados del mundo.Un ambiente con n localidades tendrá $n \times 2^n$

- Acciones: Izq, Der, Asp, (en este ejemplo) Arriba, Abajo (ambientes mas grandes)

- *Estado inicial:* Cualquier estado puede ser designado como inicial.

- *Función sucesor:* Genera los estados legales que resultan de la aplicación de las tres acciones (Izquierda, Derecha, y Aspira). Ver figura 3.3.
- *Prueba del objetivo:* Se verifica si todos los cuadros están limpios.

- *Costo de la trayectoria:* Cada paso cuesta 1, así que en este caso el costo de la trayectoria es el número de pasos en ella.

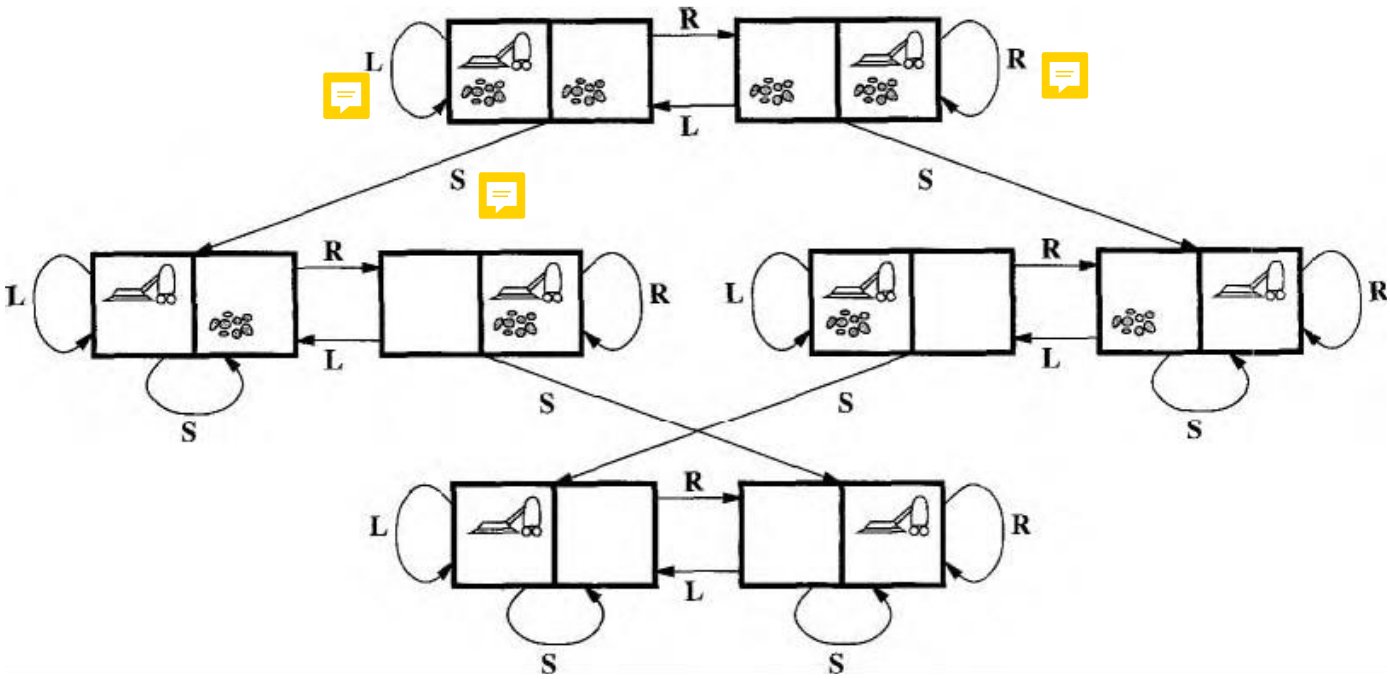
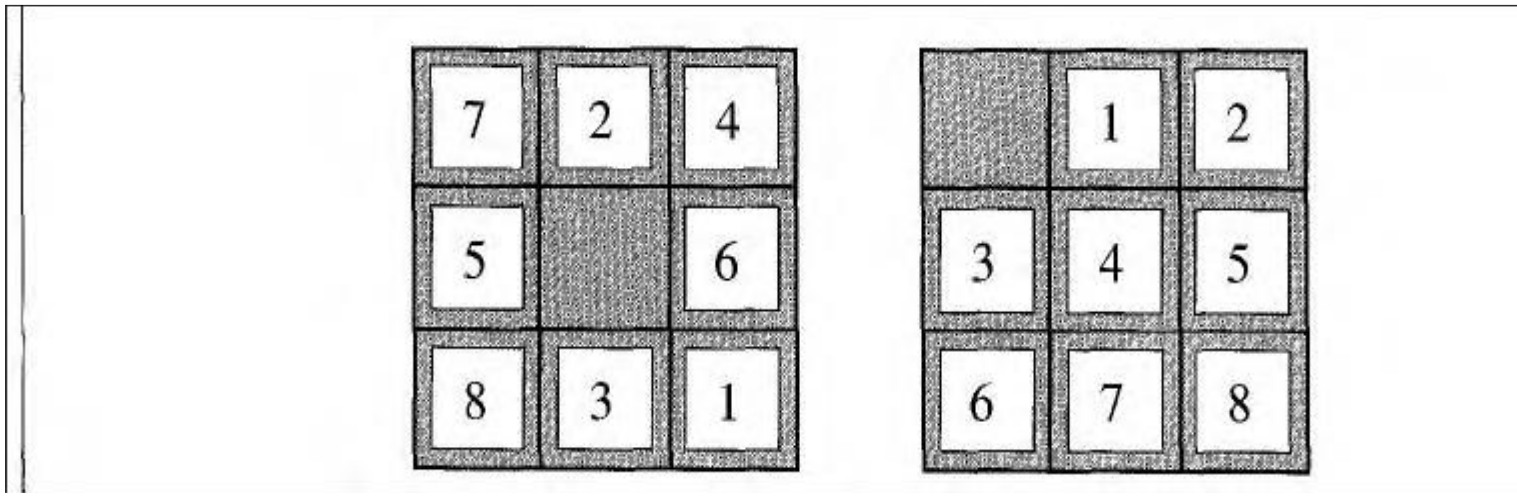


Figure 3.3 The state space for the vacuum world. Arcs denote actions: L = *Left*, R =***Right,*** S = Suck.

El problema del 8-Puzzle

Este consiste de un tablero de 3x3 con 8 cuadros numerados y un espacio en blanco. Un cuadro adyacente a este blanco puede deslizarse hacia el. El propósito es alcanzar un estado objetivo especificado, tal como el mostrado a la derecha de la figura 3.4.



Estado inicial **Estado objetivo**

Figure 3.4 A typical instance of the 8-puzzle.

Su formulación sería:

- **Estados:** Una descripción de estado que especifica la posición de cada uno de los ocho números y el blanco en uno de los nueve cuadros.
- **Estado inicial:** Cualquier estado puede ser designado como el estado inicial.
- **Función sucesor:** Esta función genera los estados legales que resultan de aplicar cuatro **acciones** (mover el blanco a la Izquierda, Derecha. Arriba, o Abajo)
- **Prueba del objetivo:** Con ello se "checa" si el estado se acopla a la configuración del objetivo mostrado en la figura 3.4 (otras configuraciones son posibles).
- **Costo de la Trayectoria:** Cada paso cuesta 1. Así que el costo de la trayectoria es el número de pasos en ella.

Esta clase de acertijos pertenece a los problemas NP-completos, de tal manera que no se espera encontrar métodos significativamente mejores, para el caso peor, que los que serán descritos en el tema de Búsquedas informadas.

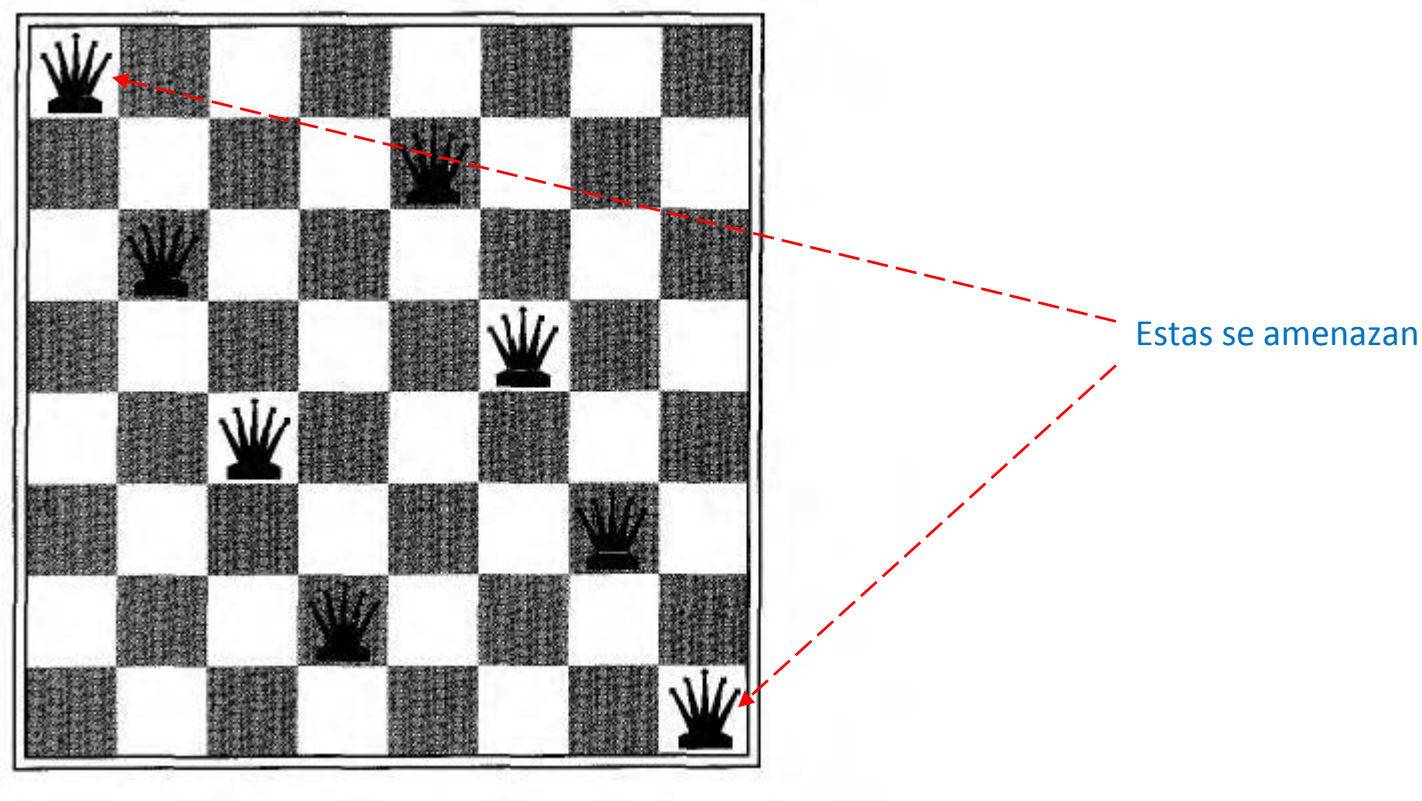
Tiene **9!/2 = 182, 440** estados que pueden alcanzarse.

El acerijo-15 (4x4) tiene **1.3 trillones de estados,** (aunque instancias aleatorias pueden ser resultas en pocos milisegundos).

El acertijo-24 (5x5) tiene alrededor de 10^{25} **estados**, e instancias aleatorias son muy difíciles de resolver óptimamente. Con lo que hay actualmente en cuanto a máquinas y algoritmos (actualmente ya toma varias horas el llegar a una solución).

El problema de la 8-Reinas

Consiste en colocar 8 reinas en un tablero de ajedrez de tal forma que no se amenacen.



Hay dos formulaciones del problema

- Formulación incremental: se inicia con un estado vacio y cada acción agrega una reina al estado.
- Formulación de estado-completo: se inicia con ocho reinas y se van moviendo alrededor.

Para la primera formulación tenemos:

- **Estados:** cualquier arreglo de 0 a 8 reinas sobre el tablero es un estado.
- **Estado inicial:** Ninguna reina sobre el tablero.

- **Acciones**: agregar una reina a cada cuadro vacio.
- **Función sucesor:** regresa el tablero con una reina agregada en el cuadro especificado.
- **Prueba del objetivo:** 8 reinas están sobre el tablero, ninguna se ataca

Hay 64! Secuencias por investigar $\approx 1.8 \times 10^{14}$ secuencias.

Una mejor formulación de esto es prohibir situar una reina en un lugar que este ya atacado:

- **Estados:** arreglos de n reinas ($0 \leq n \leq 8$), una por columna de las que están más a la izquierda, sin que se ataquen, son estados.

- **Acciones:** Agregar una reina a cualquier cuadro en la columna vacía más a la izquierda, de tal forma que no sea atacada por cualquier otra reina.

Con esto el espacio de estados se reduce a 2,057 y las soluciones son fáciles de encontrar.

Problemas del mundo-real

En la realidad los problemas pueden resultar más complicados para hacer una formulación de ellos. En este caso tenemos los problemas de encontrar rutas, los problemas de viajes de una aerolínea, los sistemas de enrutamiento en redes de computadoras, la planeación de operaciones militares, etc.

Un ejemplo simplificado de una aerolínea seria:

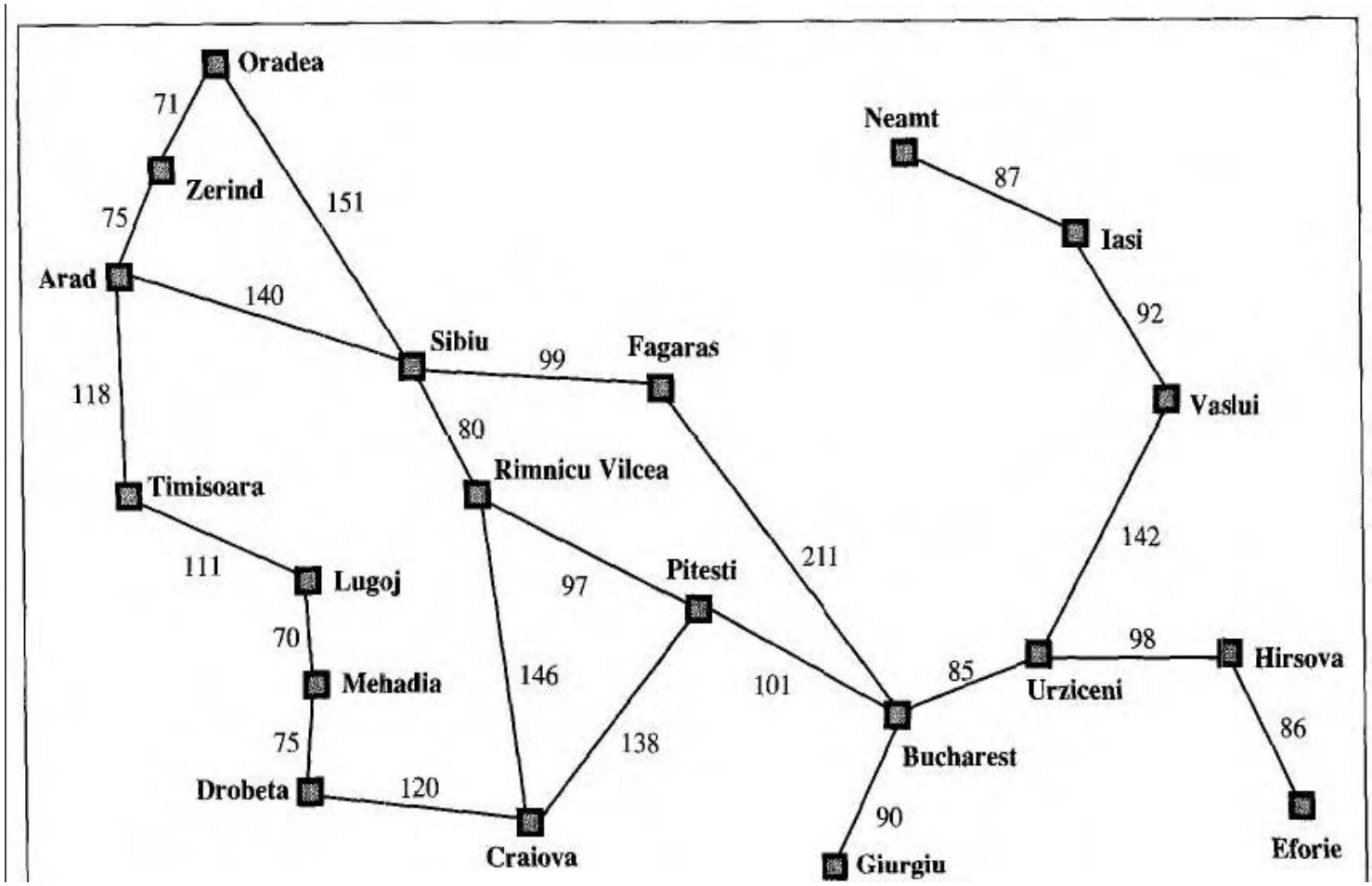
- **Estados:** Cada uno de ellos está representado por un lugar (Ejem. en el aeropuerto) y el tiempo actual, aspectos históricos
- **Estado inicial:** Es especificado por los requerimientos del usuario.
- **Acciones:** Tomar algún vuelo desde la localidad actual
- **Función sucesor:** Esta función regresa los estados resultantes desde tomar cualquier vuelo calendarizado (especificado tal vez por clase de asiento y lugar), tiempo de tránsito entre aeropuertos, etc.
- **Prueba del objetivo:** ¿Estamos en el destino en el tiempo especificado?
- **Costo de la trayectoria:** Esto depende del costo monetario, tiempo de espera, tiempo de vuelo, procedimientos de aduana e inmigración, calidad del asiento, tiempo de día, tipo de avión, recompensa por millas de viajero-frecuente, etc.

Los sistemas de agencias comerciales pueden formular así el problema, pero con más complicaciones adicionales que dependen de las políticas de cada aerolínea. Las experiencias de los que saben es, que no todo sale de acuerdo a lo planeado.

El problema del viajero es semejante al de encontrar rutas.

Ejem: el problema de "Visitar cada ciudad (figura 3.2) al menos una vez, iniciando y terminando en Bucarest"

- Las acciones corresponden a viajes entre ciudades adyacentes
- El espacio de estados debe considerar el lugar actual más el conjunto de las ciudades que se han visitado.
- El estado inicial debe ser entonces "En Bucarest; visitado {Bucarest}," un estado intermedio por ejemplo seria "En Vaslui; visitado {Bucarest, Urziceni, Vaslui},"
- Prueba del objetivo: Verificar si el agente está en Bucarest y todas las 20 ciudades han sido visitadas



El problema del agente viajero (TSP)

El TSP cae dentro del tipo anterior. Aquí cada ciudad debe ser visitada exactamente una vez. Lo que se quiere es encontrar el tour más corto. Es conocido como NP-duro. Ha habido esfuerzos por mejorar las capacidades de los algoritmos y han sido usados para atacar otros problemas, tales como la planeación de movimientos en perforaciones automáticas de tarjetas de circuitos y en máquinas de almacenamiento en pisos de compras.



function SIMPLE-PnontEm-SOLVING-AGENT(percept) returns an action persistent *seq*, an action sequence, initially empty
state, some description of the current world state *goal*, a goal, initially null
problem, a problem formulation *state UPDATE-ST ATE(state , percept)*
if *seq* is empty then
goal +(— FORMULATE-GOAL(state)
problem +— FORMULATE-PROBLEM(*state, goal) seq* SEARcii(pro*blem)*
if *seq* = *failure* **then** return a null action
action 4— FIRST(seq)
s +— REST(seq)
return *action*

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem.
searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

El problema de "grabado" en circuitos impresos (VLSI)



A VLSI **layout** problem requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield. The layout problem comes after the logical design phase, and is usually split into two parts: **cell layout** and **channel routing.** In cell layout, the primitive components of the circuit are grouped into cells, each of which performs some recognized function. Each cell has a fixed footprint (size and shape) and requires a certain number of connections to each of the other cells. The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells. Channel routing finds a specific route for each wire through the gaps between the cells. These search problems are extremely complex, but definitely worth solving. In Chapter 4, we

will see some algorithms capable of solving them.

Robot navigation is a generalization of the route-finding problem described earlier. Rather than a discrete set of routes, a robot can move in a continuous space with (in principle) an infinite set of possible actions and states. For a circular robot moving on a flat surface, the space is essentially two-dimensional. When the robot has arms and legs or wheels that must also be controlled, the search space becomes many-dimensional. Advanced techniques are required just to make the search space finite. We examine some of these methods in Chapter 25. In addition to the complexity of the problem, real robots must also deal with errors in their sensor readings and motor controls.

Automatic assembly sequencing of complex objects by a robot was first demonstrated by FREDDY (Michie, 1972). Progress since then has been slow but sure, to the point where the assembly of intricate objects such as electric motors is economically feasible. In assembly problems, the aim is to find an order in which to assemble the parts of some object. `If` the wrong order is chosen, there will be no way to add some part later in the sequence without undoing some of the work already done. Checking a step in the sequence for feasibility is a difficult geometrical search problem closely related to robot navigation. Thus, the generation of legal successors is the expensive part of assembly sequencing. Any practical algorithm must avoid exploring all but a tiny fraction of the state space. Another important assembly PROTEINDESIGN problem is **protein design,** in which the goal is to find a sequence of amino acids that will fold into a three-dimensional protein with the right properties to cure some disease.
In recent years there has been increased demand for software robots that perform

In-ternet searching, looking for answers to questions, for related information, or for shopping deals. This is a good application for search techniques, because it is easy to conceptualize the Internet as a graph of nodes (pages) connected by links. A full description of Internet search is deferred until Chapter 10

Tarea 4 (Ejercicios 3.6 y 3.7) de la 3ra. edición

3.6 Give the initial state, goal test, successor function, and cost function for each of the following. Choose a formulation that is precise enough to be implemented.
a. You have to color a planar map using only four colors, in such a way that no two adjacent regions have the same color.
b. A 3-foot-tall monkey is in a room where some bananas are suspended from the 8-foot ceiling. He would like to get the bananas. The room contains two stackable, movable, climbable 3-foot-high crates.
c. You have a program that outputs the message "illegal input record" when fed a certain file of input records. You know that processing of each record is independent of the other records. You want to discover what record is illegal.
d. You have three jugs, measuring 12 gallons, 8 gallons, and 3 gallons, and a water faucet. You can fill the jugs up or empty them out from one to another or onto the ground. You need to measure out exactly one gallon.

3.7 Consider the problem of finding the shortest path between two points on a plane that has convex polygonal obstacles as shown in Figure 3.31. This is an idealization of the problem that a robot has to solve to navigate in a crowded environment.

a.Suppose the state space consists of all positions (x, y) in the plane. How many states are there? How many paths are there to the goal?

b.Explain briefly why the shortest path from one polygon vertex to any other in the scene must consist of straight-line segments joining some of the vertices of the polygons. Define a good state space now. How large is this state space?

c.Define the necessary functions to implement the search problem, including an ACTIONS function that takes a vertex as input and returns a set of vectors, each of which maps the current vertex to one of the vertices that can be reached

in a straight line. (Do not forget the neighbors on the same polygon.) Use the straight-line distance for the heuristic function

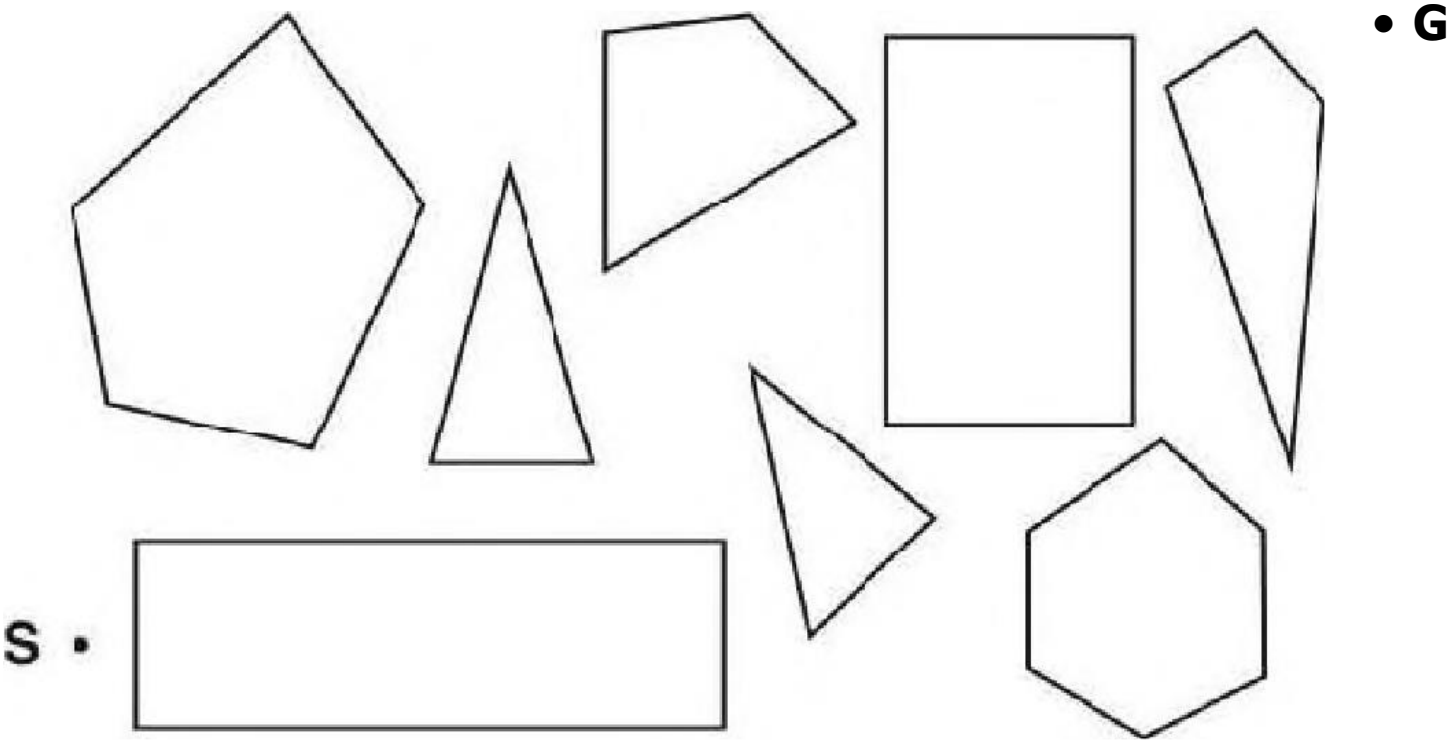
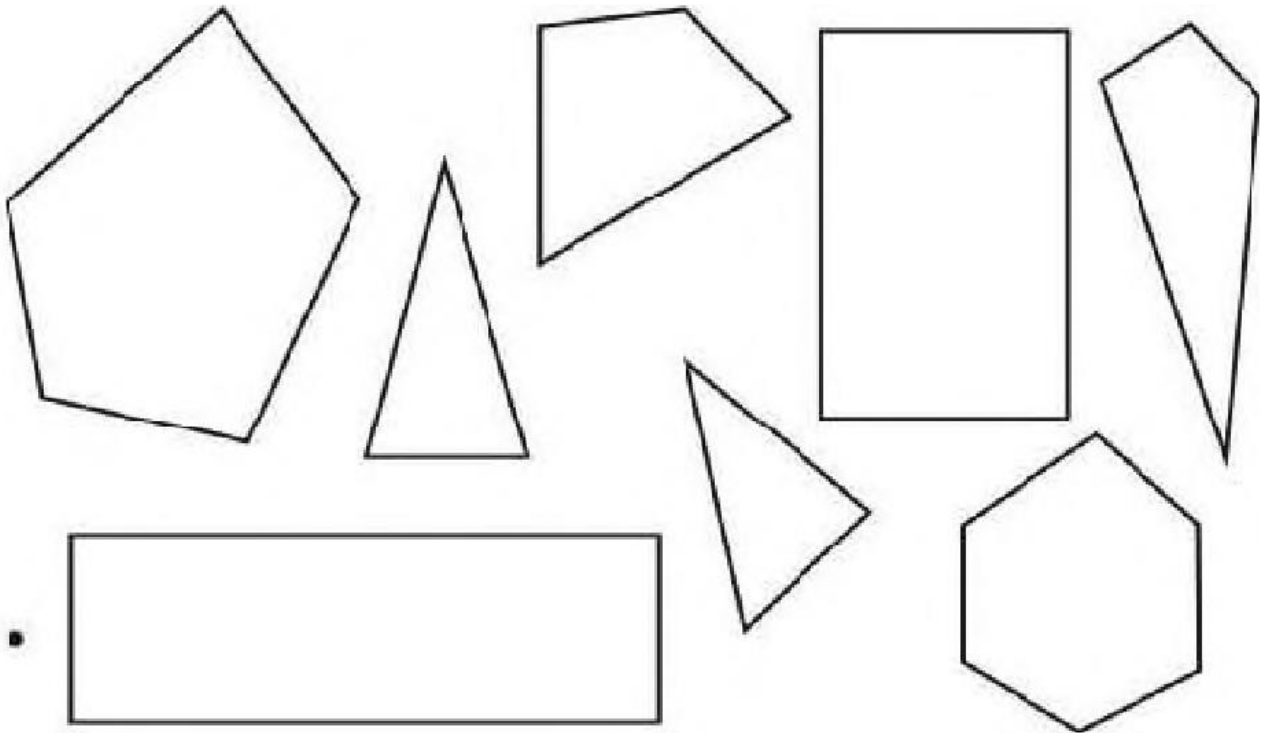


Figure 3.31 A scene with polygonal obstacles. *S* and *G* are the start and goal states



S •